DeepLearning.AI

RAG Evaluation

Retrieval Quality Metrics

Common ingredients to most retriever quality metrics:

The Prompt
The specific prompt being evaluated

Ranked Results
Documents returned in ranked order

Ground Truth
All documents labeled as relevant or irrelevant

If you want to evaluate your retriever
you need to know the correct answers

Retrieval Quality Metrics

Common ingredients to most retriever quality metrics:

The Prompt
The specific prompt being evaluated

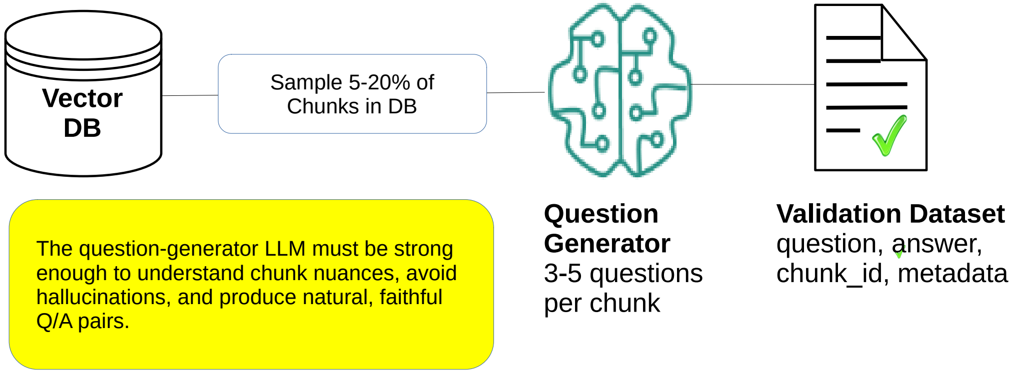
Ranked Results
Documents returned in ranked order

Ground Truth
All documents labeled as relevant or irrelevant

Which means you need a validation/test set

With questions and the chunk that contains the answer

Generating a Validation/Test set



Example:
Vector DB has 1000 chunks
Select 10%→100 chunks
Generate 5 questions per chunk
Have a validation set of 5*100=500 question/chunk pairs

*A consequence of this is that you only have 1 relevant chunk per question

Precision and Recall

Precision

Measures how many of the returned documents are relevant

```
Relevant Retrieved / Total Retrieved
```

Recall

Measures how many of the relevant documents are returned

```
Relevant Retrieved / Total Relevant
```

Example

10 Relevant Documents in Knowledge Base

First Run

Retrieved: 12 Documents

Relevant: 8 Documents

Precision (8/12) **66%**

Recall (8/10) **80%**

Second Run

Retrieved: 15 Documents

Relevant: 9 Documents

Precision (9/15) **60%** ↓

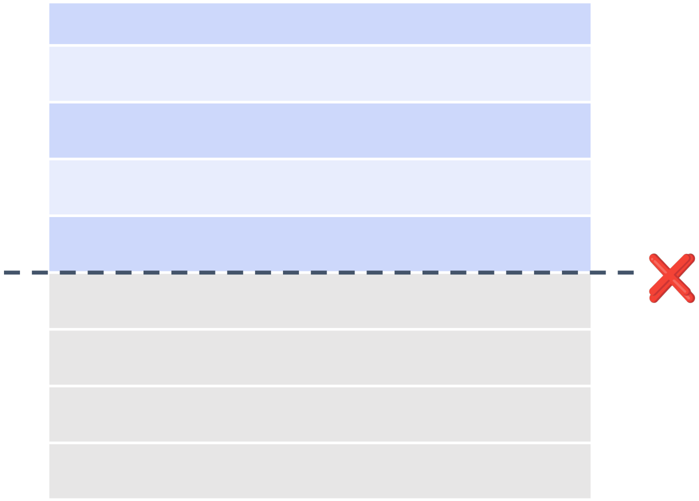
Recall (9/10) **90%** ↑

Precision penalizes for **returning irrelevant** documents

Recall penalizes for **leaving out relevant** documents

100% Precision & Recall: Rank <u>all</u> relevant documents most highly and <u>only</u> return those

Top k



Top K

- Retrieval metrics are influenced by **how many documents the retriever returns**
- Metrics are discussed in terms of **top-k** documents

Example

For a given query a vector DB has 8 relevant chunks

Precision = relevant retrieved / total retreived
Recall = relevant retrieved / total relevant

Rank	Relevant
1	relevant
2	-
3	-
4	relevant
5	-
6	relevant
7	relevant
8	-
9	relevant
10	relevant

Precision @5

2 out of 5

40%

Precision @10

6 out of 10

60%

Recall@10

6 out of 8 total relevant

75%

Top-5 or Top-1 is stricter

Top-5 to Top-15 often used

What if there is only 1 relevant chunk per query?

If it's returned in top K?

P@5   = 1/5   = .2
P@10 = 1/10 = .1

Top 5

Rank	Relevant
1	relevant
2	-
3	-
4	-
5	-
6	-
7	-
8	-
9	-
10	-

Top 10

What if there is only 1 relevant chunk per query?

If it's not returned in top K?

P@5 = 0/5 = 0
P@10 = 0/10 = 0

Rank	Relevant
1	-
2	-
3	-
4	-
5	-
6	-
7	-
8	-
9	-
10	-

Top 5

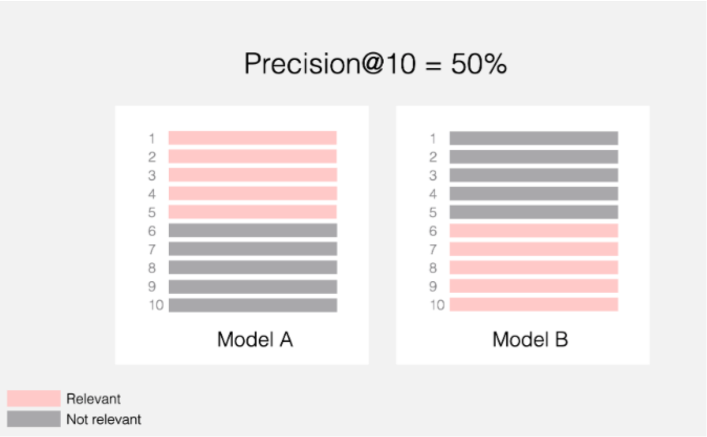
Top 10

Precision @k, Pros and Cons

Easy to explain: Precision@K is intuitive, so it's good for communicating system quality to stakeholders.

Tracks accuracy (not order): It measures how many of the top-K results are relevant, regardless of their positions.

Not rank-aware: It ignores where relevant items appear—5 relevant in positions 1–5 and 6–10 both give 50% at K=10.



Same performance
Even though left selection ordered relevant chunks higher in list

Summary: Use Precision@K when you want the accuracy of the top-K results without caring about their order.

Content from https://www.evidentlyai.com/ranking-metrics/precision-recall-at-k

Average Precision and Mean Average Precision(MAP)

MAP@K evaluates average precision for relevant documents in first K documents. It is built off a related metric called “average precision”.

Rank	Item	Precision@K
1	relevant	1/1 **1.0**
2	-	1/2 **0.5**
3	-	1/3 **0.3**
4	relevant	2/4 **0.5**
5	relevant	3/5 **0.6**
6	-	3/6 **0.5**

Rewards ranking relevant documents highly

1. **Sum precisions** for relevant docs only.

$$1 + 0.5 + 0.6 = 2.1$$

2. **Divide** by number of relevant documents

$$2.1 / 3 = 0.7$$

3. This calculation gives **Average Precision or AP**, for Mean Average Precision you find the AP value across many prompts

Example: Calculate Average Precision, K=6

Rank	Item	Precision@K
1	relevant	1/1 **1.0**
2	relevant	2/2 **1.0**
3	relevant	3/3 **1.0**
4	-	3/4 **0.75**
5	-	3/5 **0.6**
6	-	3/6 **0.5**

1. **Sum precisions** for relevant docs only.

   $1 + 1 + 1 = 3$

2. **Divide** by number of relevant documents

   $3 / 3 = 1$

Reciprocal rank

Measures the rank of the first relevant document in the returned list

`Reciprocal Rank = 1 / Rank`

First relevant at rank 1 — 1.0

First relevant at rank 2 — 0.5

First relevant at rank 3 — 0.33

No relevant docs = 1/∞ — 0.0

The later the first relevant document appears, the worse the reciprocal rank

Mean Reciprocal Rank (MRR) averages over many prompts

Mean Reciprocal Rank

Search 1	**First relevant at rank 1**	1.0
Search 2	**First relevant at rank 3**	0.33
Search 3	**First relevant at rank 6**	0.17
Search 4	**First relevant at rank 2**	0.5

1. **Sum all ranks**

   1.0 + 0.33 + 0.17 + 0.5 = 2.0

2. **Divide by number of searches**

   2.0 / 4

   MRR = 0.5

How to use retriever metrics

Recall or recall@K

Most cited metric, captures fundamental goal of finding relevant documents

Precision & MAP

Asses irrelevant documents and ranking effectiveness

Mean Reciprocal Rank

How well model performs at the very top of ranking

Metrics help:

- Evaluate retriever performance
- Check if adjustments improve results

All metrics depend on having ground truth relevant documents